

Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **ATM Machine Design**.

In this article, we'll guide you through a clear step-by-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- Each user has one bank account at the bank
- User should be able to insert card and perform transactions
- ATM should authenticate user on the basis of the PIN entered
- Once authenticated, user should be able to perform transactions: view balance, deposit cash, withdraw cash
- User can perform only one transaction at any given time (per session)
- At the end of a transaction, appropriate success/failure messages should be displayed
- Card should be returned when user exits
- Admin should be able to refill cash, audit inventory, take ATM in/out of service, and view logs

Interview Tip: Always ask the interviewer to confirm the requirements before jumping into code. It shows clarity and alignment.

Edge Cases to Consider:

- Wrong PIN attempts
 - Any withdrawal limit?
 - How to ensure ATM maintains balance consistency? (Lock or last moment check)
 - What happens in power failure?
 - Notes mismatch while depositing?
 - Handle floating point errors?
-

Step 2: Identify Core Entities

1. ATM (Core Entity)

- id: String [PK]
- location: String
- isOnline: boolean
- currentState: ATMState
- cashDrawer: CashDrawer
- currentSession: Session

2. Card

- id: String [PK]
- accountId: String [FK]
- expiry: String (MM/YY)
- isBlocked: boolean
- pinRetriesLeft: int

3. Account

- id: String [PK]
- holderName: String
- balanceMinorUnits: long
- dailyWithdrawalLimitMinor: long
- dailyWithdrawalUsedMinor: long
- isActive: boolean

4. Transaction

- id: String [PK]
- atmId: String [FK]
- sessionId: String [FK]
- accountId: String [FK]
- type: TransactionType (WITHDRAW, DEPOSIT, BALANCE)
- amountMinorUnits: long
- status: TransactionStatus (PENDING, SUCCESS, FAILED)
- dispensedNotes: Map<Denomination, Integer>
- depositedNotes: Map<Denomination, Integer>
- createdAt: long
- timeoutAt: long

5. CashDrawer

- atmId: String [FK]
- notesByDenomination: Map<Denomination, Integer>

6. Denomination (Enum)

- 500
- 200
- 100

7. TransactionType (Enum)

- WITHDRAW
- DEPOSIT

- BALANCE

8. TransactionStatus (Enum)

- PENDING
- SUCCESS
- FAILED

9. AdminUser

- id: String [PK]
- name: String
- pinHash: String
- isActive: boolean

10. Session

- id: String [PK]
- atmId: String [FK]
- cardId: String [FK]
- accountId: String [FK]
- startTime: long
- endTime: long
- isActive: boolean
- currentTransactionId: String [FK] (nullable)
- transactionType: String
- amount: long

Interview Tip: Clearly identifying entities and enums at this stage helps define relationships (like ATM ↔ Session, Card ↔ Account) and ensures smooth implementation of transactions, authentication, and admin operations.

Step 3: Visualize Interaction Flows

1. Card Operations:

- Insert card
- Eject card
- Authenticate card

2. Session Operations:

- Start session
- End session

3. Transaction Operations:

- Show balance
- Withdraw cash
- Deposit cash
- Acknowledge transaction

4. ATM Operations:

- Take offline
- Bring online
- Audit cash

5. Admin Operations:

- Login admin
- Refill cash
- Audit cash

6. State-Driven Flow:

- States orchestrate services and auto-transition
 - Invalid operations throw exceptions
 - Controllers handle exceptions and return responses
-

Step 4: Defines Class Structures & Relationships

Layers of Architecture

We will design the **architecture layers** of the system in a structured way, ensuring **separation of concerns** and **modularity**. The system will be organized into the following layers:

Client/UI Layer → Controller Layer → Domain Layer → Service Layer

CONTROLLERS:

1. ATMController (ATM Management)

- - void takeOffline(String atmId)
- - void bringOnline(String atmId)
- - CashDrawer auditCash(String atmId)

2. CardController (Card Operations)

- - boolean insertCard(String cardId)
- - void ejectCard(String atmId)
- - boolean authenticateCard(String cardId, String pin)

3. SessionController (Session Management)

- - Session startSession(String atmId, String cardId)
- - void endSession(String sessionId)

4. TransactionController (Transaction Operations)

- - Transaction showBalance(String sessionId)
- - Transaction withdrawCash(String sessionId, long amountMinorUnits)

- - Transaction depositCash(String sessionId, Map<Denomination, Integer> notes)
- - void acknowledgeTransaction(String transactionId)

5. AdminController (Admin Operations)

- - boolean loginAdmin(String adminId, String pin)
 - - void refillCash(String atmlId, Map<Denomination, Integer> notes)
 - - CashDrawer auditCash(String atmlId)
-

SERVICES:

1. ATMService (ATM Management)

- - void takeOffline(String atmlId)
- - void bringOnline(String atmlId)
- - CashDrawer auditCash(String atmlId)

2. CardService (Card Operations)

- - boolean insertCard(String atmlId, String cardId)
- - void ejectCard(String atmlId)
- - boolean validateCard(String cardId)
- - boolean authenticateCard(String cardId, String pin)

3. SessionService (Session Management)

- - Session startSession(String atmlId, String cardId)
- - void endSession(String sessionId)
- - Session getCurrentSession(String atmlId)
- - void handleSessionTimeout(String sessionId)

4. TransactionService (Transaction Operations)

- - Transaction showBalance(String sessionId)

- - Transaction withdrawCash(String sessionId, long amountMinorUnits)
- - Transaction depositCash(String sessionId, Map<Denomination, Integer> notes)
- - void acknowledgeTransaction(String transactionId)
- - boolean validateTransaction(Transaction transaction)

5. AdminService (Admin Operations)

- - boolean loginAdmin(String adminId, String pin)
- - void refillCash(String atmId, Map<Denomination, Integer> notes)
- - CashDrawer auditCash(String atmId)

REPOSITORIES:

1. ATMRepository

- - ATM save(ATM atm)
- - Optional<ATM> findById(String atmId)
- - List<ATM> findAll()
- - void updateATMState(String atmId, ATMState state)

2. CardRepository

- - Card save(Card card)
- - Optional<Card> findById(String cardId)
- - void updatePinRetries(String cardId, int retriesLeft)
- - void blockCard(String cardId)

3. AccountRepository

- - Account save(Account account)
- - Optional<Account> findById(String accountId)
- - void updateBalance(String accountId, long newBalance)

- - void updateDailyWithdrawalUsed(String accountId, long amountUsed)

4. TransactionRepository

- - Transaction save(Transaction transaction)
- - Optional<Transaction> findById(String transactionId)
- - List<Transaction> findBySession(String sessionId)
- - List<Transaction> findByATMAndTimeRange(String atmId, long startTime, long endTime)
- - void updateTransactionStatus(String transactionId, TransactionStatus status)

5. CashDrawerRepository

- - CashDrawer save(CashDrawer cashDrawer)
- - Optional<CashDrawer> findByATMId(String atmId)
- - void updateCashInventory(String atmId, Map<Denomination, Integer> notes)

6. SessionRepository

- - Session save(Session session)
- - Optional<Session> findById(String sessionId)
- - Optional<Session> findActiveByATM(String atmId)
- - void endSession(String sessionId)

7. AdminUserRepository

- - AdminUser save(AdminUser adminUser)
- - Optional<AdminUser> findById(String adminId)
- - boolean validateAdminCredentials(String adminId, String pinHash)

STATE PATTERN IMPLEMENTATION:

1. ATMState (State Interface)

- + void insertCard(ATM atm, String cardId) throws InvalidATMOperationException
- + void ejectCard(ATM atm) throws InvalidATMOperationException
- + void enterPin(ATM atm, String pin) throws InvalidATMOperationException
- + void selectTransaction(ATM atm, TransactionType type) throws InvalidATMOperationException
- + void processTransaction(ATM atm, long amount) throws InvalidATMOperationException
- + void endSession(ATM atm) throws InvalidATMOperationException
- + ATMState next(ATM atm)

2. Concrete States:

- IdleState, CardInsertedState, AuthenticatedState, TransactionSelectedState, TransactionCompletedState, OutOfServiceState
 - Each state delegates operations to Services and sets next() for auto-transition
-

STRATEGY PATTERN IMPLEMENTATION:

1. TransactionStrategy (Strategy Interface)

- + Transaction processTransaction(String sessionId, long amount, Map<Denomination, Integer> notes)

2. Concrete Strategies:

- WithdrawalStrategy
- DepositStrategy
- BalanceInquiryStrategy

Step 5: Core Use Cases & Methods

1. Card Insertion Use Case:

- POST /api/atm/{atmId}/insert-card →
- ATMController.insertCard() →
- ATM.currentState.insertCard(cardId) →
- [State orchestrates: CardService.validateCard(), SessionService.startSession()] →
- Auto next() → CardInsertedState → Return success

2. PIN Authentication Use Case:

- POST /api/session/{sessionId}/authenticate →
- CardController.authenticateCard() →
- ATM.currentState.enterPin(pin) →
- [State orchestrates: CardService.authenticateCard()] →
- Auto next() → AuthenticatedState → Return authentication result

3. Balance Inquiry Use Case:

- GET /api/session/{sessionId}/balance →
- TransactionController.showBalance() →
- ATM.currentState.selectTransaction(BALANCE) →
ATM.currentState.processTransaction(0) →
- [State orchestrates: TransactionService (BalanceInquiryStrategy)]
→

- Create transaction record → Auto next() → TransactionCompletedState → Return balance
-

4. Cash Withdrawal Use Case:

- POST /api/session/{sessionId}/withdraw →
 - TransactionController.withdrawCash() →
 - ATM.currentState.selectTransaction(WITHDRAW) →
ATM.currentState.processTransaction(amountMinorUnits) →
 - [State orchestrates: TransactionService.withdrawCash()
(WithdrawalStrategy) → Validate amount → Check balances →
Calculate notes → Update account & ATM cash → Create
transaction record → Dispense cash] →
 - Auto next() → TransactionCompletedState → Return transaction
result
-

5. Cash Deposit Use Case:

- POST /api/session/{sessionId}/deposit →
- TransactionController.depositCash() →
- ATM.currentState.selectTransaction(DEPOSIT) →
ATM.currentState.processTransaction(totalAmountFromNotes) →
- [State orchestrates: TransactionService.depositCash()
(DepositStrategy) → Validate notes → Count cash → Update
account & ATM cash → Create transaction record] →
- Auto next() → TransactionCompletedState → Return transaction
result

6. Card Ejection Use Case:

- POST /api/atm/{atmId}/eject-card →
- CardController.ejectCard() →
- ATM.currentState.ejectCard() →
- [State orchestrates: SessionService.endSession(); hardware eject] →
- Auto next() → IdleState → Return success

7. Admin Login Use Case:

- POST /api/admin/login →
- AdminController.loginAdmin() →
- AdminService.loginAdmin() →
- Validate credentials → Create admin session → Return login result

8. Cash Refill Use Case:

- POST /api/admin/atm/{atmId}/refill →
- AdminController.refillCash() →
- AdminService.refillCash() →
- Validate admin session → Update cash inventory → Log refill operation → Return success

9. ATM Status Change Use Case:

- POST /api/admin/atm/{atmId}/status →
 - ATMController.updateATMStatus() →
 - ATMService.takeOffline() or ATMService.bringOnline() →
 - Update ATM state → Eject card if offline → Return success
-

10. Transaction Acknowledgment Use Case:

- POST /api/transaction/{transactionId}/acknowledge →
 - TransactionController.acknowledgeTransaction() →
 - TransactionService.acknowledgeTransaction() →
 - Update transaction status → Return success
-

Step 6: Apply OOP Principles & Design Patterns

Design Patterns Used:

- **State Pattern** – for managing ATM operational states
 - **Repository Pattern** – for data access abstraction
 - **Service Layer Pattern** – for business logic separation
 - **Strategy Pattern** – for different transaction processing strategies
 - **Factory Pattern** – for creating different transaction types
 - **Command Pattern** – for encapsulating transaction operations
-

OOP Principles Applied:

- **Single Responsibility** – each class has one clear purpose
- **Open/Closed** – easy to extend with new transaction types or states
- **Liskov Substitution** – all state implementations are interchangeable

- **Interface Segregation** – focused interfaces for different concerns
 - **Dependency Inversion** – high-level modules depend on abstractions
 - **Encapsulation** – domain objects hide internal implementation details
 - **Polymorphism** – different transaction types handled uniformly
-

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Wrong PIN attempts** – Track retries in Card entity; block after 3 failed attempts; clear count on success
 - **Withdrawal limits** – Validate daily limit, ATM cash, transaction max, and denominations; use greedy algorithm
 - **ATM balance consistency** – Use database transactions, optimistic locking, rollback on failure, maintain audit trail
 - **Power failure handling** – Graceful shutdown, eject card, persist state, recover on restoration, complete pending transactions
 - **Notes mismatch during deposit** – Validate denominations, count notes, reject if mismatch, log discrepancies
 - **Floating point errors** – Use integer minor units, round display, BigDecimal for critical calculations
 - **Invalid operation for current state** – Throw `InvalidATMOperationException`; map to error response; state remains unchanged
-

Implementation Strategies:

- **State Management** – Use State pattern for clean state transitions

- **Concurrency** – Synchronized methods for thread-safe operations
 - **Persistence** – Database transactions for data consistency
 - **Error Handling** – Exception-first, rollback on failures
 - **Logging** – Detailed audit trail for all operations
 - **Recovery** – Persist state for power failure recovery
 - **Validation** – Multi-layer validation for data integrity
 - **Security** – PIN hashing and secure authentication
 - **Monitoring** – Real-time monitoring of ATM status and transactions
 - **Maintenance** – Admin interface for system management
-

Step 8: Package Structure

settings.json ×

1

2

3

Summary

This simplified ATM design focuses on 10 core API flows perfect for a 60-minute interview:

- **Card Insertion:** POST /api/atm/{atmId}/insert-card (start session & validate card)
- **PIN Authentication:** POST /api/session/{sessionId}/authenticate (verify user)
- **Balance Inquiry:** GET /api/session/{sessionId}/balance (check account balance)

- **Cash Withdrawal:** POST /api/session/{sessionId}/withdraw (dispense cash & update balances)
 - **Cash Deposit:** POST /api/session/{sessionId}/deposit (deposit cash & update account)
 - **Card Ejection:** POST /api/atm/{atmId}/eject-card (end session & eject card)
 - **Admin Login:** POST /api/admin/login (authenticate admin)
 - **Cash Refill:** POST /api/admin/atm/{atmId}/refill (update ATM cash inventory)
 - **ATM Status Change:** POST /api/admin/atm/{atmId}/status (bring ATM online/offline)
 - **Transaction Acknowledgment:** POST /api/transaction/{transactionId}/ack (confirm transaction completion)
-

Key Features Implemented:

- State-based ATM operation handling (CardInserted, Authenticated, TransactionCompleted, Idle)
- Strategy pattern for transaction types: Withdrawal, Deposit, Balance Inquiry
- Thread-safe operations and concurrency handling
- Persistent transaction logging and audit trail
- Multi-layer validation for security, cash limits, and denominations
- Graceful handling of power failures and recovery
- Optimistic locking for concurrent transactions
- Admin operations for ATM refill and status management
- Exception-first error handling for invalid operations
- Clean separation of controllers, services, and state logic

Design Pattern Highlights:

- **State Pattern** – Manages ATM states and transitions (CardInserted, Authenticated, Idle, TransactionCompleted)
- **Strategy Pattern** – Different transaction processing strategies (Withdraw, Deposit, Balance)
- **Repository Pattern** – Abstracts data access for accounts, transactions, and ATM cash
- **Service Layer Pattern** – Separates business logic from controllers
- **Factory Pattern** – Creates transaction objects dynamically
- **Command Pattern** – Encapsulates transaction execution logic
- **Polymorphism** – Different transaction types handled uniformly via strategy interfaces